# SIMPLE DATA FILTERING USING PYTHON

## 1. Introduction

Simple Data Filtering is a Python-based project designed to select relevant student records from a collection based on specified conditions. The program filters students by marks, age, and department.

The project introduces basic data-processing techniques used to extract useful information from structured datasets.

## 2. Objective

The objectives of this project are:

- To understand how structured data can be represented in Python.
- To filter records using conditional statements.
- To select students based on marks, age, and department.
- To combine multiple filtering conditions.
- To practise functions, list comprehensions, and logical operators.

## 3. Problem Statement

Student datasets may contain many records, and identifying students who satisfy particular requirements can be time-consuming when done manually.

This project automates the process by filtering student records according to user-defined conditions and displaying the matching results.

## 4. Tools and Technologies Used

**Python 3:** Used to implement the filtering logic.

**Lists and Dictionaries:** Used to store student records and their attributes.

**Conditional Statements:** Used to evaluate filtering criteria.

**List Comprehensions:** Used to create filtered collections efficiently.

**Visual Studio Code:** Used for development and execution.

**GitHub:** Used to store the source code and documentation.

## 5. Dataset Description

The project uses eight manually created fictional student records.

Each record contains:

- Student ID
- Name
- Age
- Department
- Marks

This structure allows the program to access individual student attributes and apply filtering conditions.

## 6. Methodology

The project follows these steps:

1. Create a collection of student records.
2. Store each student as a dictionary.
3. Display a menu of filtering options.
4. Accept the user's selected operation.
5. Apply the selected condition to the records.
6. Combine conditions when required.
7. Display matching records in a tabular format.
8. Handle invalid input and cases with no matching records.

## 7. Filtering Operations

### 7.1 Filtering by Marks

The program accepts a minimum marks threshold and selects students whose marks are greater than or equal to that value.

### 7.2 Filtering by Age

The program accepts a maximum age and selects students younger than that age.

### 7.3 Filtering by Department

The program selects students belonging to a specified department. Department matching is case-insensitive.

### 7.4 Filtering by Multiple Conditions

The program combines minimum marks and maximum age using the logical AND operator. A student is selected only when both conditions are satisfied.

## 8. Sample Dataset

| Student ID | Name | Age | Department | Marks |
|---|---|---:|---|---:|
| S101 | Arun | 20 | Computer Science | 85 |
| S102 | Priya | 21 | Data Science | 92 |
| S103 | Rahul | 19 | Artificial Intelligence | 76 |

| S104 | Sneha | 22 | Computer Science | 68 |
| S105 | Kiran | 20 | Data Science | 81 |
| S106 | Ananya | 18 | Electronics | 95 |
| S107 | Vikram | 23 | Computer Science | 74 |
| S108 | Meera | 21 | Artificial Intelligence | 88 |

## 9. Results and Output

The program displays the complete student dataset or a filtered subset based on the selected criteria.

For example:

- Filtering by marks of at least 80 returns five students.
- Filtering for age below 21 returns four students.
- Filtering for Data Science returns two students.
- Filtering for marks of at least 80 and age below 21 returns two students.

**Screenshot Evidence:** Insert a screenshot of the actual program output here.

## 10. Testing and Validation

The program should be tested with different marks thresholds, age limits, department names, and combinations of conditions.

Testing should also cover invalid numeric input, invalid menu choices, unknown departments, and conditions that return no matching records.

Record the actual test results before marking each test as passed.

## 11. Concepts Learned

This project demonstrates:

- Lists and dictionaries in Python
- Structured data representation
- Conditional statements
- Comparison operators
- Logical operators
- List comprehensions
- Functions
- Input validation
- Data filtering

## 12. Limitations and Future Improvements

The current project uses a small, manually created dataset. It does not import external datasets or save modifications to persistent storage.

Future improvements could include:

- Loading student records from CSV or Excel files.
- Filtering datasets using Pandas.
- Adding sorting and summary statistics.
- Creating an interactive dashboard.
- Exporting filtered results to CSV.

## 13. Conclusion

The Simple Data Filtering project demonstrates how Python can extract relevant records from structured data using conditional logic.

It provides practical experience in filtering data and lays the foundation for more advanced data analysis using libraries such as Pandas.

## 14. Project Links

GitHub Repository: https://github.com/venky1318/simple-data-filtering